

JAVA 17 OCP · YDS STUDY SERIES

Unit 09 · Collections and Generics — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge Java 17/OCP odağında hazırlanmış **özgün çalışma soruları** içerir; gerçek sınav sorusu değildir. Önerilen süre 20–25 dakikadır. Her soruda önce collection contract'ını veya generic bound'u belirle.

Bu sette **8 özgün soru** vardır; her soruda aksi belirtilmedikçe tek doğru seçenek seçilir.

Sorular

Soru 1

Aşağıdaki kodun çıktısı nedir?

```
import java.util.ArrayList;
import java.util.List;

public class RemoveQuiz {
    public static void main(String[] args) {
        List<Integer> numbers = new ArrayList<>(List.of(1, 2, 3));
        numbers.remove(1);
        System.out.print(numbers);
    }
}
```

- A. [1, 2]
- B. [1, 3]
- C. [2, 3]
- D. Kod derlenir fakat `UnsupportedOperationException` fırlatır.

Soru 2

Aşağıdaki kodun çıktısı nedir?

```
import java.util.Comparator;
import java.util.TreeSet;

public class OrderingQuiz {
    record Item(int id) {}

    public static void main(String[] args) {
        var items = new TreeSet<Item>(
            Comparator.comparingInt((Item item) -> item.id() % 2));
        items.add(new Item(1));
        items.add(new Item(3));
        System.out.print(items.size());
    }
}
```

- A. 0
- B. 1
- C. 2
- D. İkinci `add()` çağırısı `IllegalArgumentException` fırlatır.

Soru 3

Wildcard kullanımı hakkında doğru olan **iki seçeneği** seçin.

- A. `List<? super Integer>` içine bir `Integer` eklenebilir.
- B. `List<? extends Number>` içinden okunan değer her zaman `Integer` variable'a atanabilir.
- C. `List<? extends Number>` içine güvenli bir `Integer` eklenebilir.
- D. `List<?>` içinden okunan her non-null değer `Object` olarak alınabilir.
- E. `List<Number>` variable'a doğrudan `new ArrayList<Integer>()` atanabilir.

Soru 4

Bir `Map<String, Integer>` içinde `"x"` key'i açıkça `null` value ile eşleştirilmiş olsun. Şu çağrı yapıldığında ne olur?

```
map.merge("x", 5, Integer::sum);
```

- A. `Integer::sum`, `null` ve `5` ile çağrılır; `NullPointerException` oluşur.
- B. Existing value `null` olduğu için key absent gibi ele alınır ve `"x"` value değeri `5` olur.
- C. `merge()` yalnız key map'te hiç yoksa çalışır; mapping değişmez.
- D. Kod derlenmez; `merge()` `null` value içeren map'lerde kullanılamaz.

Soru 5

Java 17 liste oluşturma yöntemleri için doğru ifade hangisidir?

- A. `List.of(1, 2)` üzerinde `set(0, 9)` kullanılabilir.
- B. `Arrays.asList(1, 2)` üzerinde `add(3)` kullanılabilir.
- C. `Arrays.asList(1, 2)` fixed-size'dır; `set()` destekler fakat `add()` ve `remove()` desteklemez.
- D. `new ArrayList<>(List.of(1, 2))` unmodifiable bir liste üretir.

Soru 6

Şu teknik cümlelerin doğal Türkçe çevirisi ve doğru çıkarımı hangisidir?

An upper-bounded wildcard is useful when a method only reads values from a structure, whereas a lower-bounded wildcard is useful when it needs to add values to that structure.

- A. "Upper-bounded wildcard, method yapıdan yalnız değer okuduğunda; lower-bounded wildcard ise yapıya değer eklemesi gerektiğinde kullanışlıdır." `? extends` okuma, `? super` yazma yönünü destekler.
- B. "Upper bound her tür değer eklenmesine izin verir; lower bound yalnız okumaya izin verir." PECS yönü ters çevrilmiştir.
- C. "İki wildcard türü de `List<Number>` ile tamamen aynıdır." Invariance ortadan kalkar.
- D. "Wildcard yalnız runtime type kontrolü için kullanılır." Compile-time generic uyumluluğuyla ilgisi yoktur.

Soru 7 · Map'te null ve absent ayrımı

Aşağıdaki tam Java 17 programı için doğru sonuç hangisidir? **Bir seçenek seçin.**

```
import java.util.HashMap;

public class MapAbsenceQuiz {
    public static void main(String[] args) {
        var map = new HashMap<String, Integer>();
        map.put("x", null);
        System.out.print(map.getDefault("x", 9) + ":");
        map.merge("x", 2, Integer::sum);
        map.merge("x", 3, (oldValue, newValue) -> null);
        System.out.print(map.containsKey("x"));
    }
}
```

- A. 9:false
- B. null:true
- C. Kod derlenir, ilk merge çağrısında NullPointerException oluşur.
- D. null:false

Soru 8 · Lower bound'dan güvenli okuma

Aşağıdaki tam Java 17 programı için doğru sonuç hangisidir? **Bir seçenek seçin.**

```
import java.util.ArrayList;
import java.util.List;

public class LowerBoundReadQuiz {
    public static void main(String[] args) {
        List<? super Integer> values = new ArrayList<Number>();
        values.add(3);
        Integer first = values.get(0);
        System.out.print(first);
    }
}
```

- A. Başarıyla derlenir ve 3 yazar.
- B. Kod derlenmez; `values.add(3)` geçersizdir.
- C. Kod derlenmez; okunan değer doğrudan Integer'a atanamaz.
- D. Başarıyla derlenir fakat ClassCastException fırlatır.

Cevaplar ve açıklamalar

Soru 1 — B

- **A yanlış:** `remove(1)` value 1'i değil, index 1'deki value'yu kaldırır.
- **B doğru:** Primitive `int` argument nedeniyle `remove(int index)` overload'u seçilir ve 2 silinir.
- **C yanlış:** İlk element kaldırılmamıştır.
- **D yanlış:** Dıştaki `ArrayList` mutable'dır; `List.of()` yalnız constructor input'u olarak kullanılmıştır.

Soru 2 — B

- **A yanlış:** İlk element sete eklenir.
- **B doğru:** Comparator hem 1 hem 3 için 1 üretir; comparison sonucu 0 olduğundan `TreeSet` ikinci item'ı duplicate kabul eder.
- **C yanlış:** Sorted set uniqueness kararında `equals()` yerine ordering sonucunu kullanır.
- **D yanlış:** Duplicate add exception üretmez; `false` döndürür.

Soru 3 — A ve D

- **A doğru:** Lower-bounded wildcard için güvenli yazma tipi `Integer` ve subtype'larıdır.
- **B yanlış:** Liste `Double` veya başka bir `Number` subtype'ı tutuyor olabilir; güvenli okuma tipi `Number`dir.
- **C yanlış:** Exact element type bilinmediği için non-null değer eklenemez.
- **D doğru:** Wildcard'ın exact türü bilinmese de bütün reference type'lar `Object` subtype'ıdır.
- **E yanlış:** Generics invariant'tır; `ArrayList<Integer>`, `List<Number>`'ın subtype'ı değildir.

Soru 4 — B

- **A yanlış:** Existing value `null` ise remapping function ilk birleşmede çağrılmaz.
- **B doğru:** `merge()` absent veya null-mapped key için verilen non-null value'yu doğrudan yerleştirir.
- **C yanlış:** Method hem absent hem present mapping'leri işlemek üzere tasarlanmıştır.
- **D yanlış:** Bu, `merge()` ün tanımlı runtime davranışdır; compile-time hata değildir.

Soru 5 — C

- **A yanlış:** `List.of()` sonucu unmodifiable'dır; `set()` de `UnsupportedOperationException` üretir.
- **B yanlış:** `Arrays.asList()` fixed-size view döndürür; boyut değiştiren `add()` desteklenmez.
- **C doğru:** Existing element `set()` ile değiştirilebilir, fakat listenin boyutu değiştirilemez.
- **D yanlış:** Mutable `ArrayList` constructor'ı elementleri kopyalar; sonuç üzerinde `add()`, `set()` ve `remove()` kullanılabilir.

Soru 6 — A

- **A doğru:** `whereas` iki kullanım yönünü karşılaştırır. Bu, PECS kısaltmasının “producer extends, consumer super” mantığıdır.
- **B yanlış:** Okuma/yazma yönleri ters verilmiştir.
- **C yanlış:** Wildcard, invariance'ı silmez; kontrollü bir assignment/API esnekliği sağlar.
- **D yanlış:** Generic bound'lar esas olarak compile-time type safety sağlar.

Soru 7 — D

Başarıyla derlenir. `getOrDefault`, anahtar mevcut olduğundan null döndürür; A bu davranışı merge ile karıştırır. İlk merge null eşlemeyi 2 ile doldurur ve `Integer::sum` çağrılmaz, dolayısıyla C yanlıştır. İkinci merge'in remapping sonucu null olduğu için anahtar silinir; B yanlıştır. Çıktı `null:false` olur.

Soru 8 — C

? `super Integer` güvenli Integer eklemeyi sağlar; fakat okumada garanti edilen tür `Object`'tir. B yanlıştır: ekleme geçerlidir. A derleyicinin gerçek listedeki son eklemeyi izleyerek wildcard'ı `Integer`'a daralttığını varsayar; D yanlıştır çünkü kod derlenmez. `Object first = values.get(0)` değişikliğiyle program derlenip 3 yazar.